

Introduction to Linked list

What's a Reference ?

- Like a Hotel Room number
- The room number isn't the room itself - it's just a way to find the room!
- This is NOT storing a person, it's storing a "room number" (memory address)

Person pakya = new Person("Pakya");

- null: The Empty Reference

// Like a sticky note with no room number written on it

Person nobody = null;

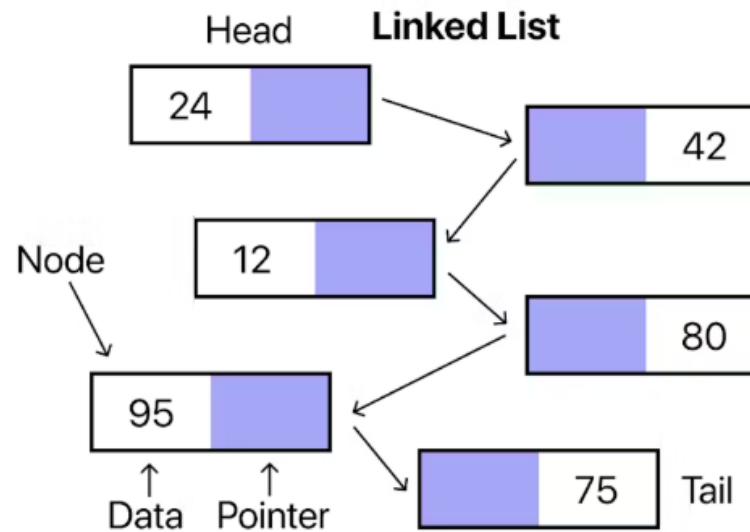
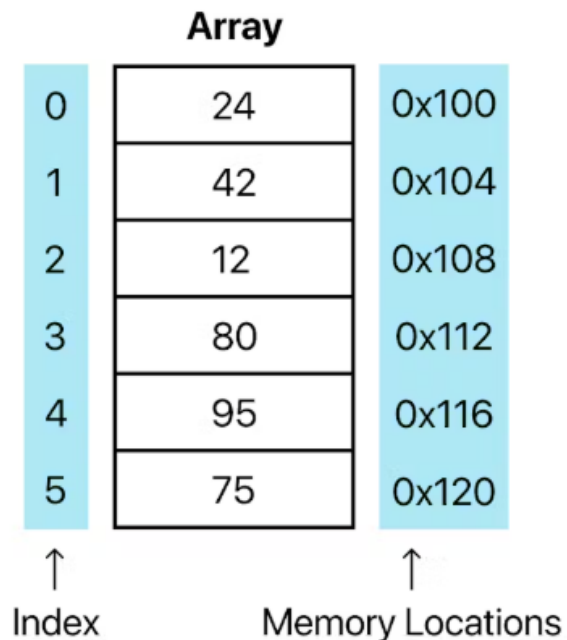
References in Action

```
class Box {  
    int value;  
    Box nextBox; // This can hold a "room number" to another Box!  
}  
  
Box first = new Box();  
first.value = 10;  
first.nextBox = null; // No next box yet  
  
Box second = new Box();  
second.value = 20;  
second.nextBox = null;  
  
// Now connect them!  
first.nextBox = second; // first now "points to" second  
  
System.out.println(first.nextBox.value); // Prints 20!
```

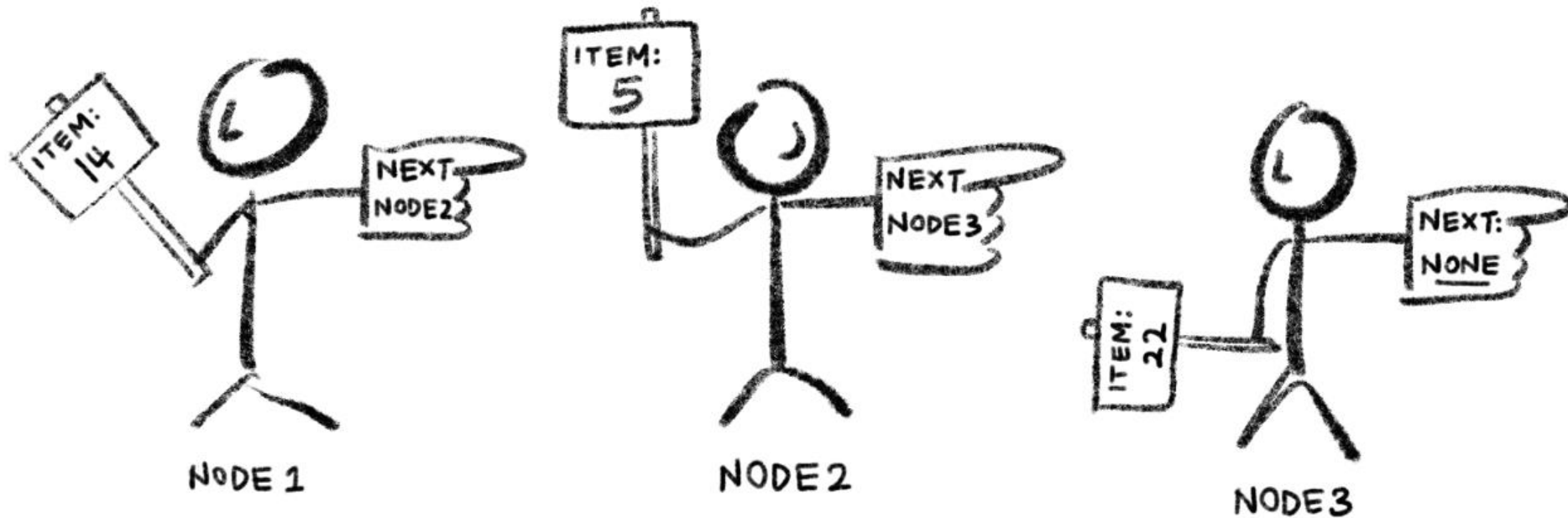
first [value: 10 | nextBox: •] -----> second [value: 20 | nextBox: null]
The `•` represents a reference (an arrow pointing to the next box).

What is a Linked list ?

- Linked list is a linear data structure which is dynamic
 - Each element points to the next
 - Order not given by their physical placement in memory

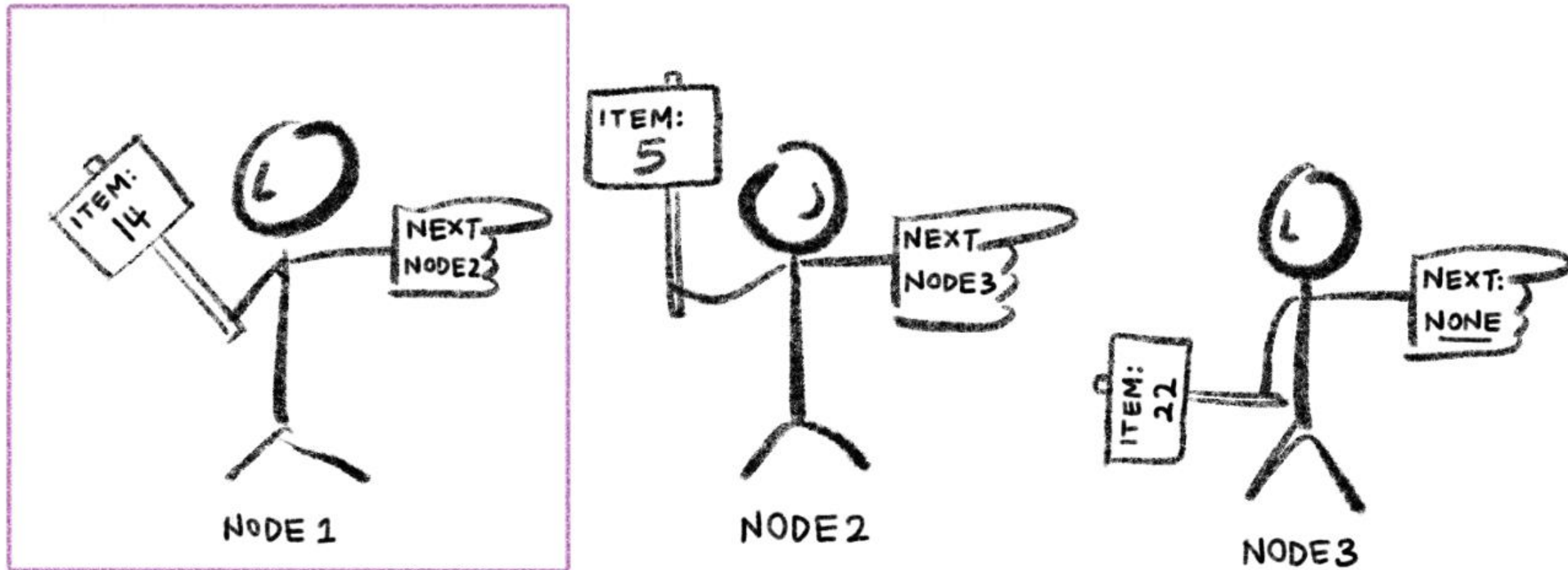


Linked List consists of Nodes

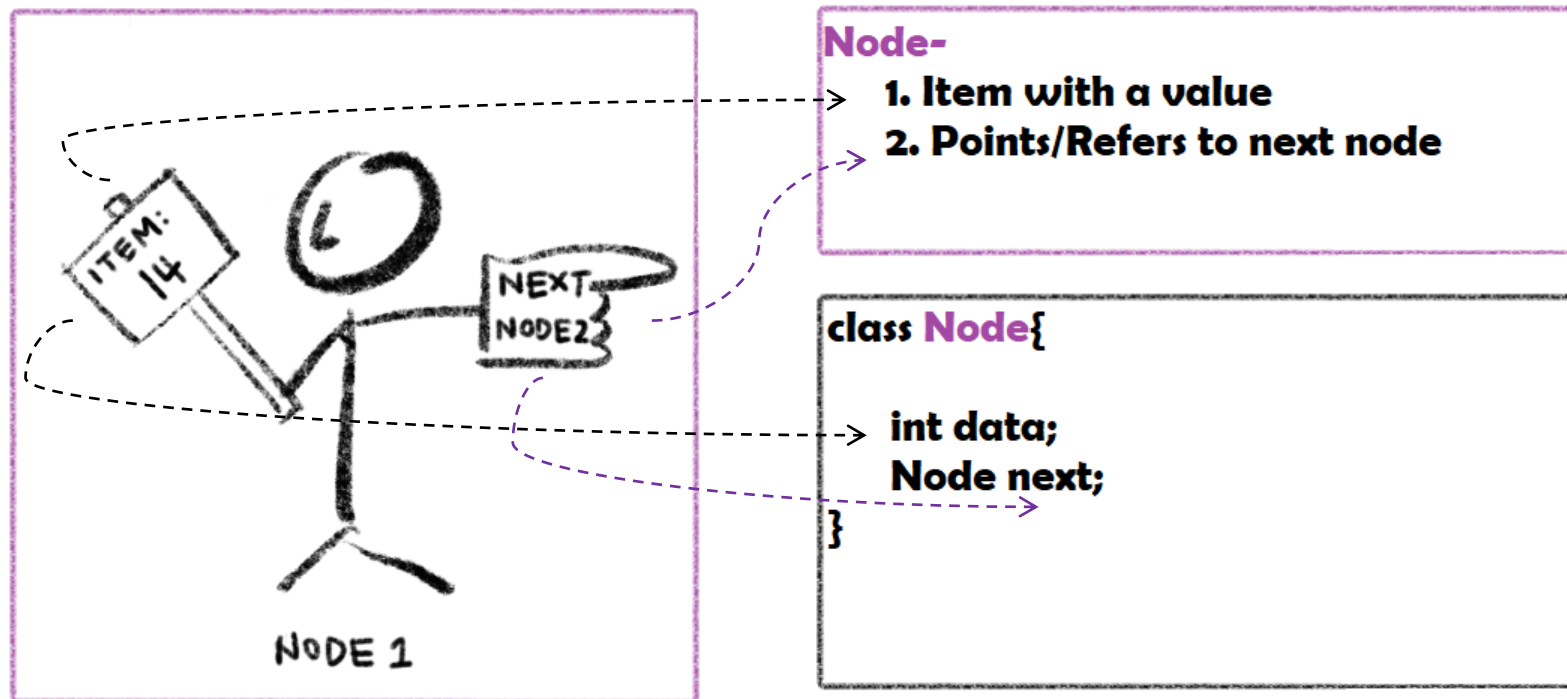


A Linked list consists of a collection of nodes which represents a sequence

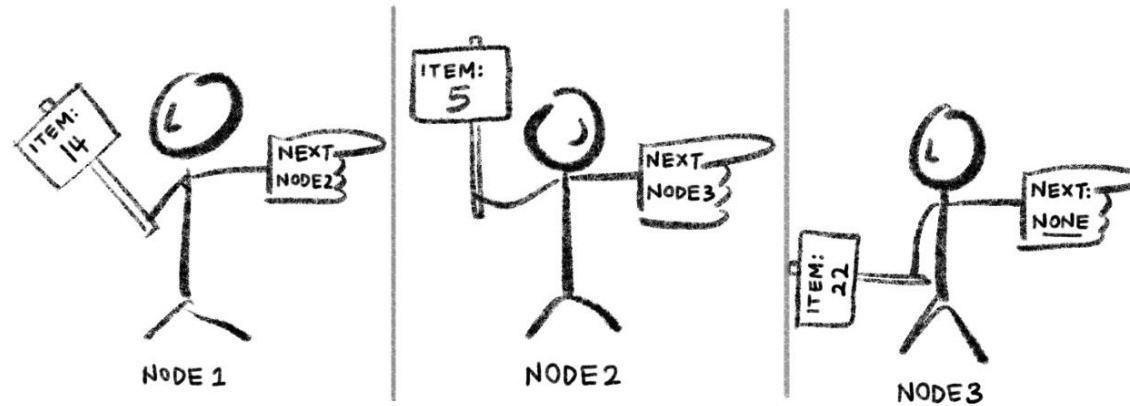
Let's focus on a single Node



What does **each Node** hold ?



What is 'Node next' ?



Each **Node** holds 2 parts :

1. An item with a value of 14 in this example, 14 is of type 'int'
2. **Refers** to the next '**Node**'. What is this type ? Well, it is a **Node**. Hence '**Node next**'. - **Reference** to the next node. So here, '**next**' is the variable name and '**Node**' is data type.

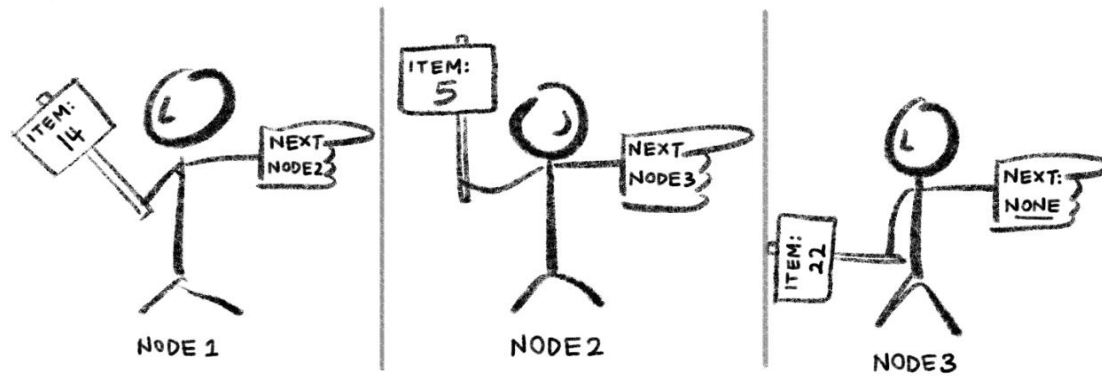
```
class Node{  
    int data;           // Holds a value  
    Node next;         // Refers to next node  
}
```


A class can contain a reference of another class

```
class Node{  
    int data;           // Holds a value  
    Node next;         // Refers to next node  
}
```

```
class A{  
    int data;  
    B b;  
}  
class B{  
    ...  
}
```

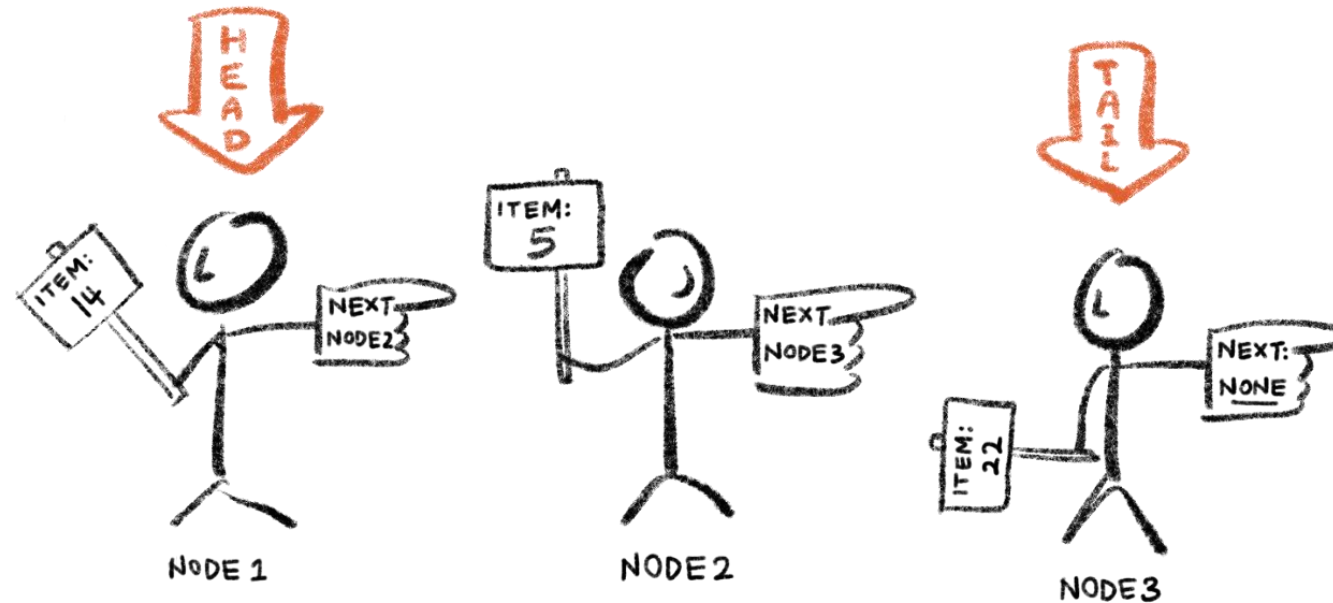
What value will 'next' contain ?



```
class Node{  
    int data;           // Holds a value  
    Node next;         // Refers to next node  
}
```

'next' will simply contain the reference/address of next node -
Address where next clue/treasure can be found

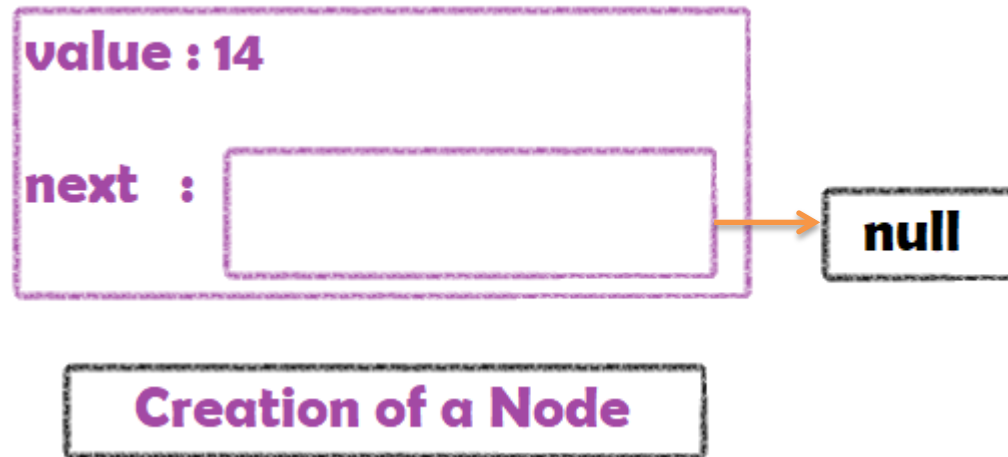
Creating a Linked List



```
class Node{  
    int data;           // Holds a value  
    Node next;         // Refers to next node  
}
```

1. `Node node= new Node();`
2. Set the value(**data**) that the node should contain
3. Set up the reference/pointer (**next**)

The first node in the Linked List



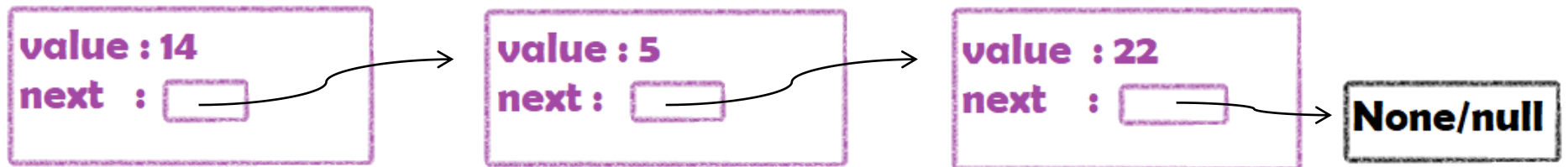
```
class Node {  
  
    int data;  
    Node next;  
  
    //Constructor  
    public Node(int data) {  
        this.data= data;  
        this.next = null;  
    }  
}  
  
Node node = new Node(14);
```

Creating a Linked List

First Step



Final Step

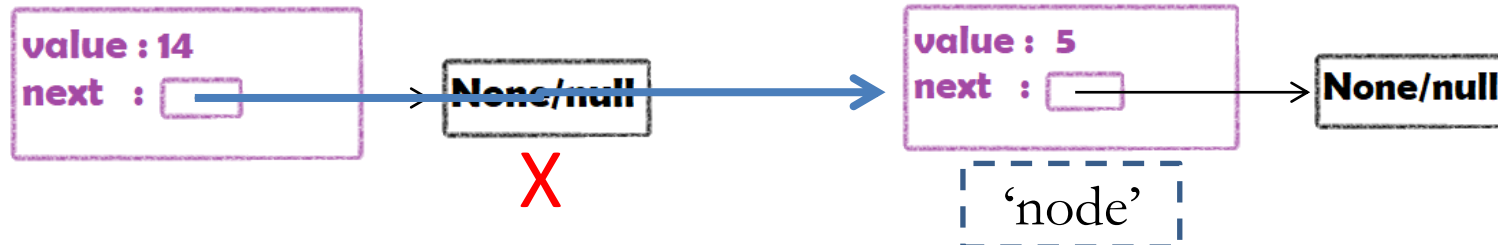


Inserting a node *at the end* in a Linked List

1. Create a new node, it's ***next*** will point to nothing/null
2. Make the existing last node point/refer to the new node

First Node

New Node



```
firstNode.next = node;    ?  
secondNode.next = node;  ?
```

```
lastElement.next = node;
```

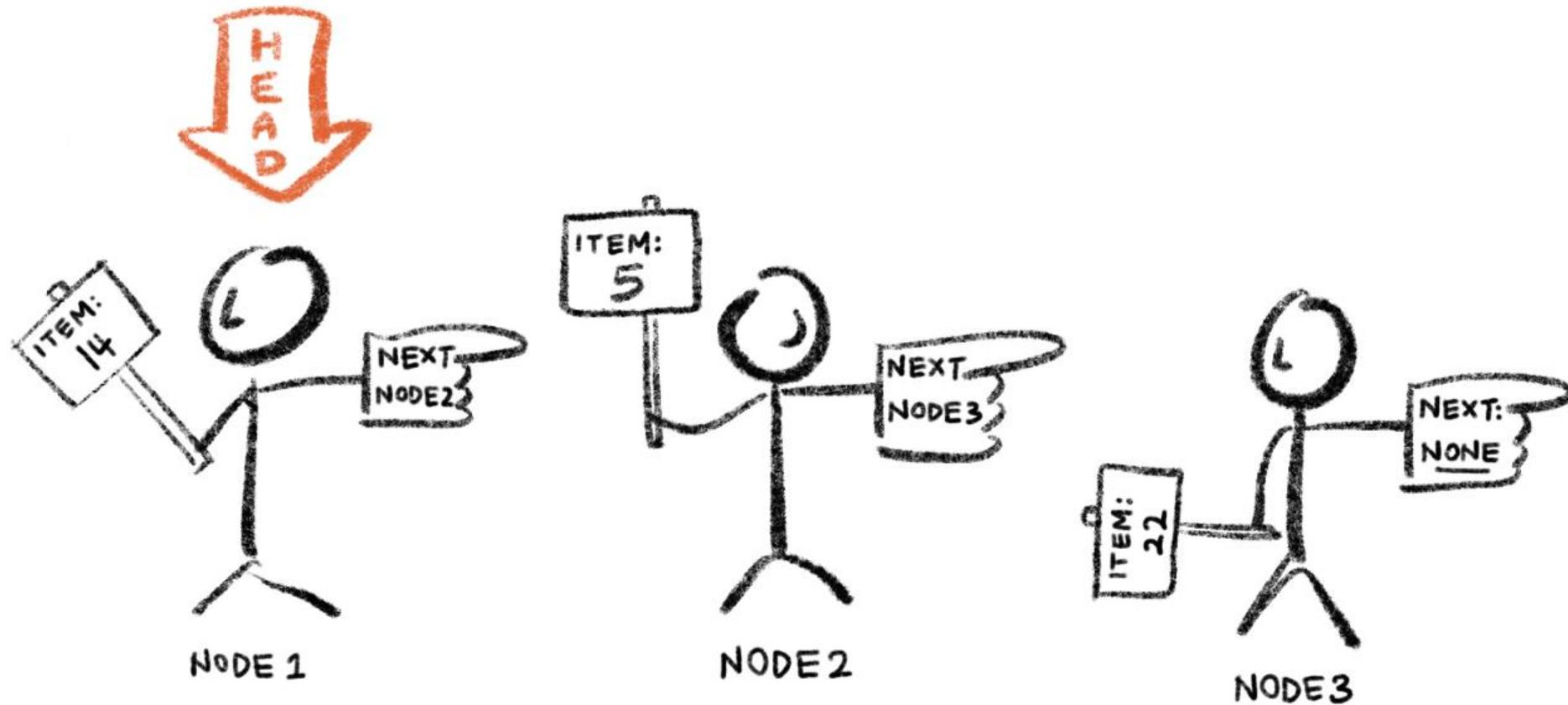
```
Node node = new Node(5)
```

```
class Node {  
  
    int data;  
    Node next;  
  
    //Constructor  
    public Node(int data) {  
        this.data= data;  
        this.next = null;  
    }  
}
```

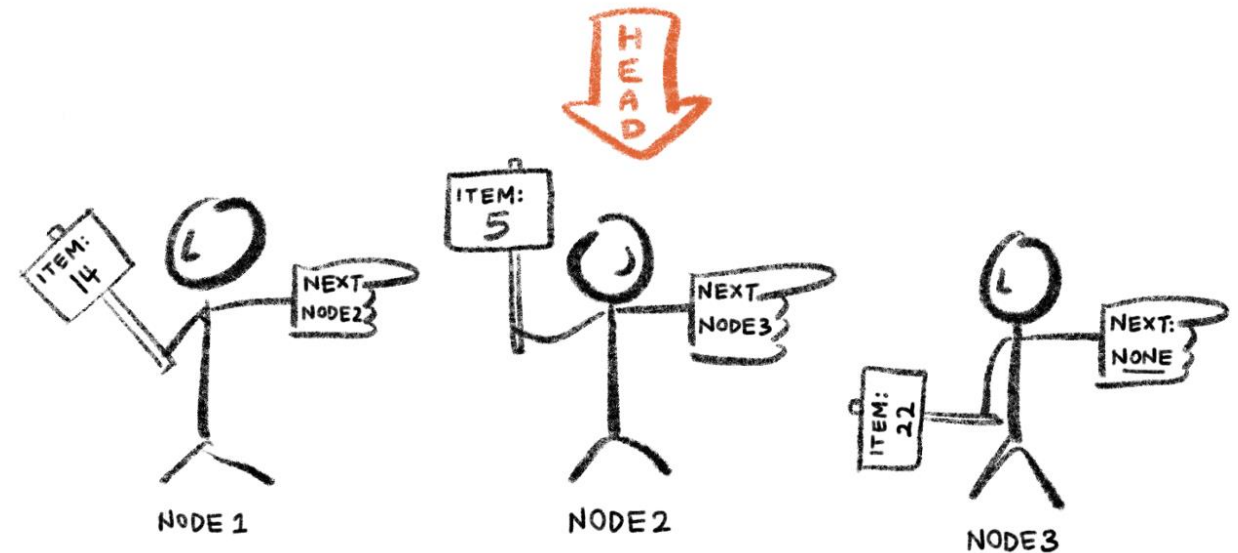
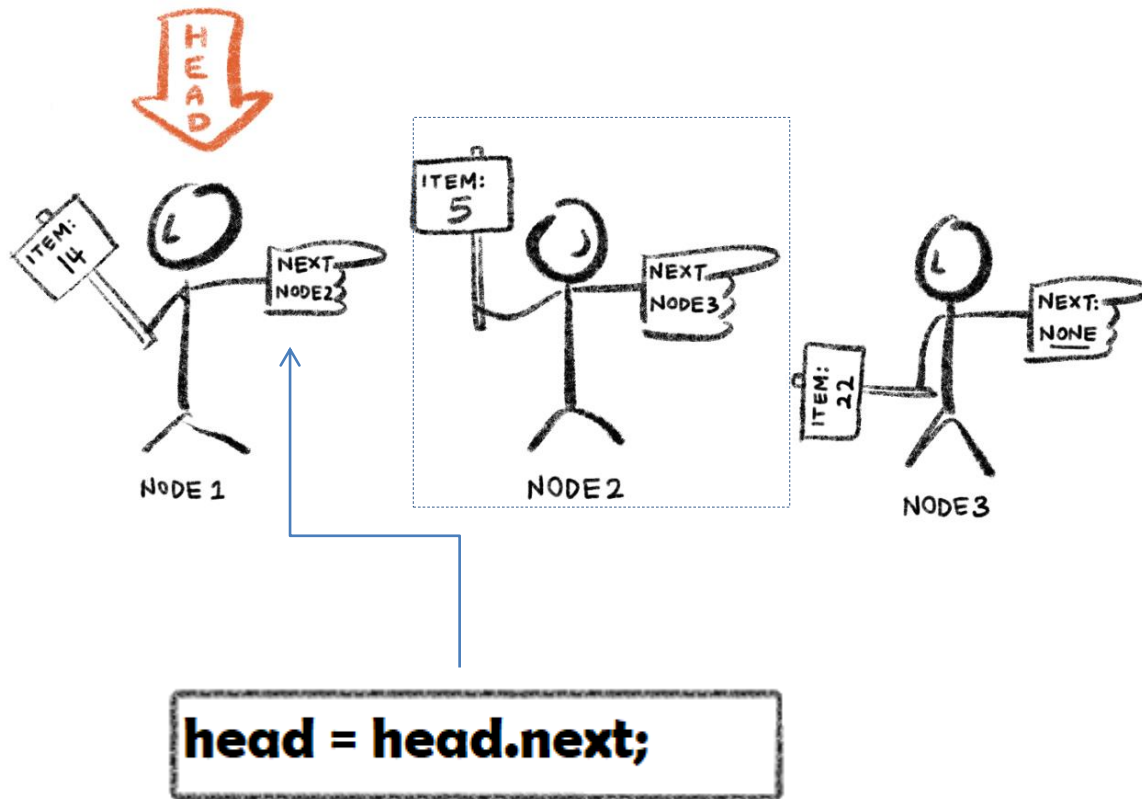
```
Node node = new Node(5);
```

How do we find the last element in the linked list ?

Remember the starting point of the treasure hunt ?

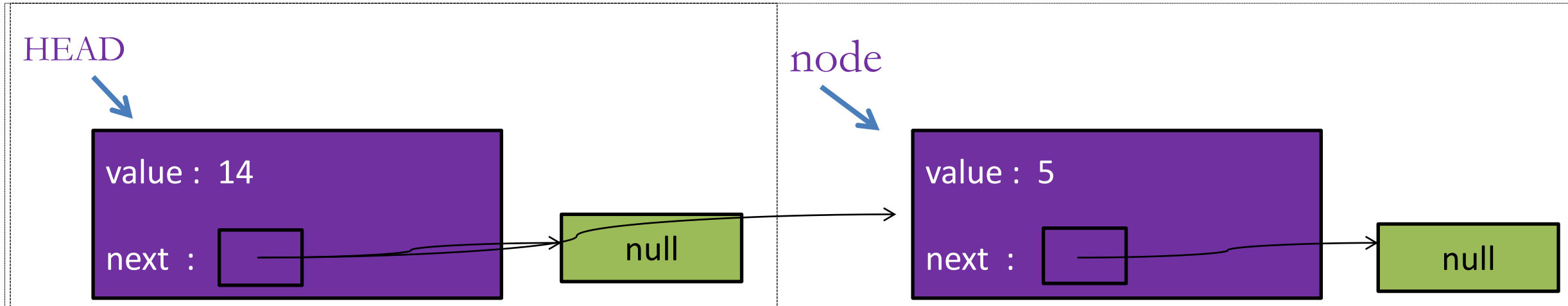


HEAD – Starting point of the treasure hunt



X By moving HEAD to point to the next element until it reaches None or null ?

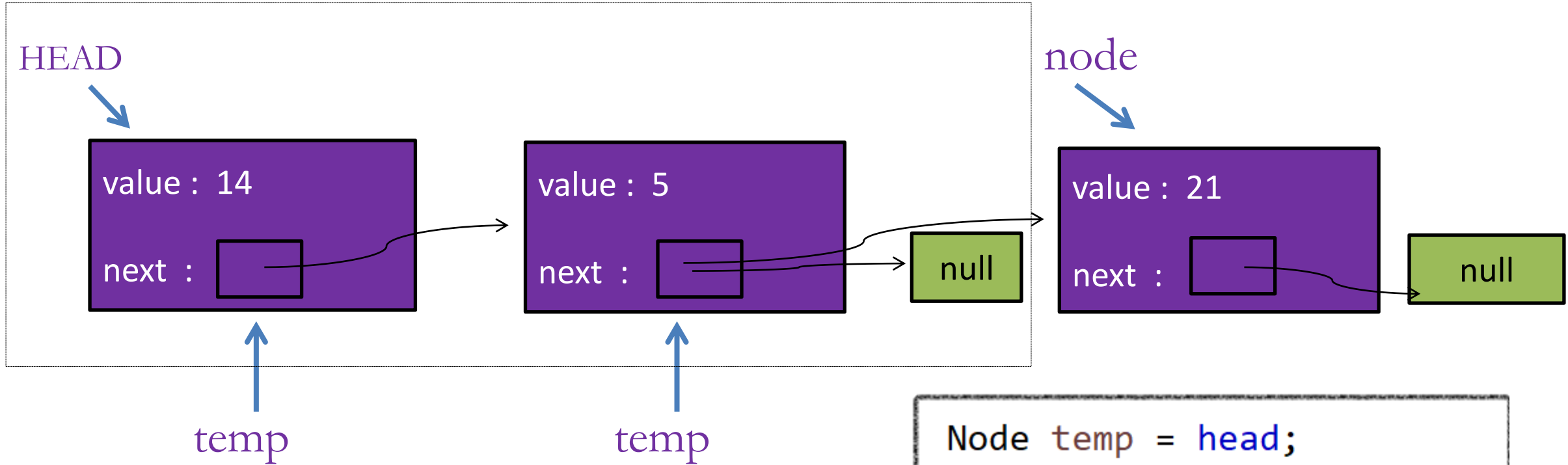
Using a temporary variable starting from HEAD



temp

```
Node temp = head;  
while(temp.next != null) {  
    temp = temp.next;  
}  
temp.next = node;
```

Using a temporary variable starting from HEAD



```
Node temp = head;  
while(temp.next != null) {  
    temp = temp.next;  
}  
temp.next = node;
```

Linked Lists: Memory Layout & Operations

- **STACK:**

- Stores primitive values (int, double, boolean, etc.)
- Stores references (memory addresses) to objects
- Memory is automatically freed when variable goes out of scope
- **Last In, First Out (LIFO)** structure
- Smaller in size, faster access

- **HEAP:**

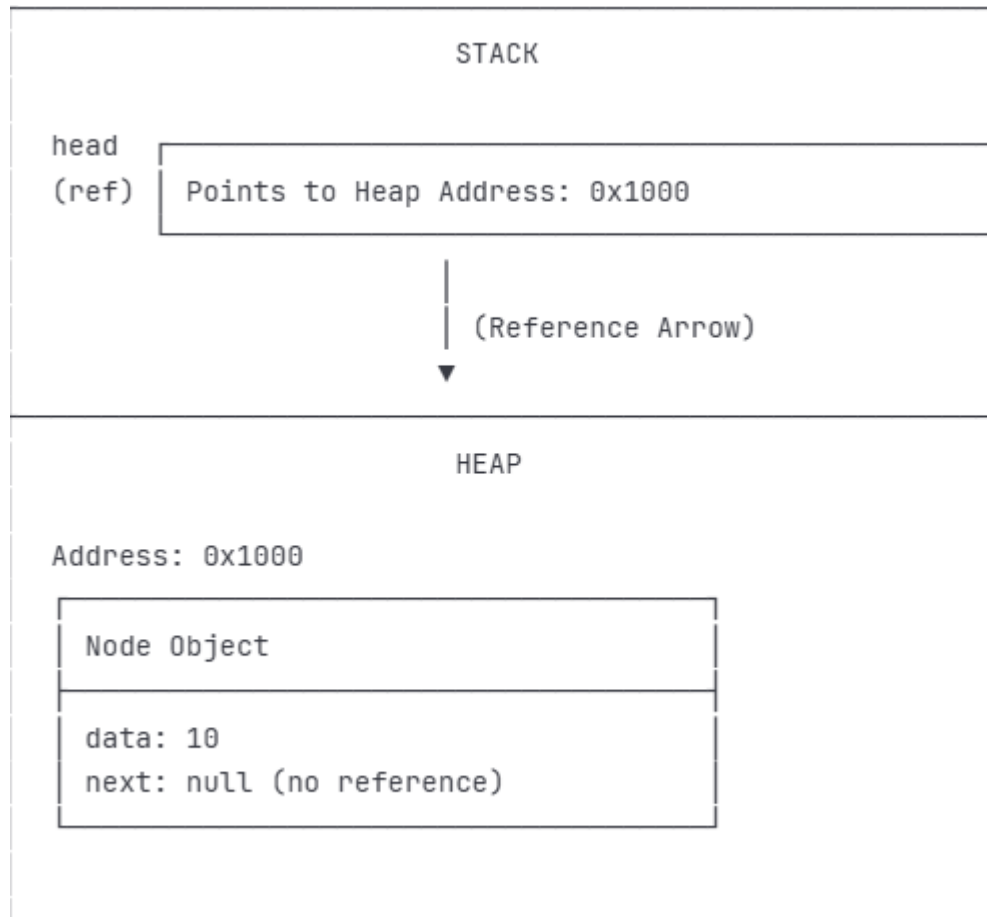
- Stores actual objects
- Memory persists until Garbage Collector removes it
- Accessed indirectly through references stored on Stack
- Larger in size, requires garbage collection

References in Java

- `Node reference = new Node(5);`
 - `reference` → variable name (on STACK) containing a memory address
 - `new Node(5)` → actual Node object created (on HEAP)
 - The arrow between them represents the reference relationship

```
class Node {  
    int data;           // actual value  
    Node next;         // reference to next node  
}
```

Creating a Single Node



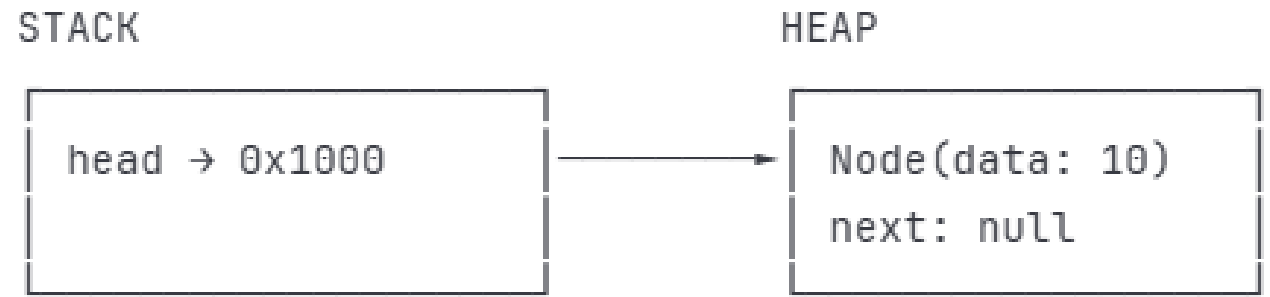
```
Node head = new Node();
head.data = 10;
head.next = null;
```

- `head` is a variable on the **STACK** storing the address `0x1000`
- The actual **Node** object lives on the **HEAP** at address `0x1000`
- `next` is `null` because there's no next node yet
- When this reference goes out of scope, the **Node** becomes eligible for garbage collection

Adding Nodes to a SLL

- Step 1 (Create first node)

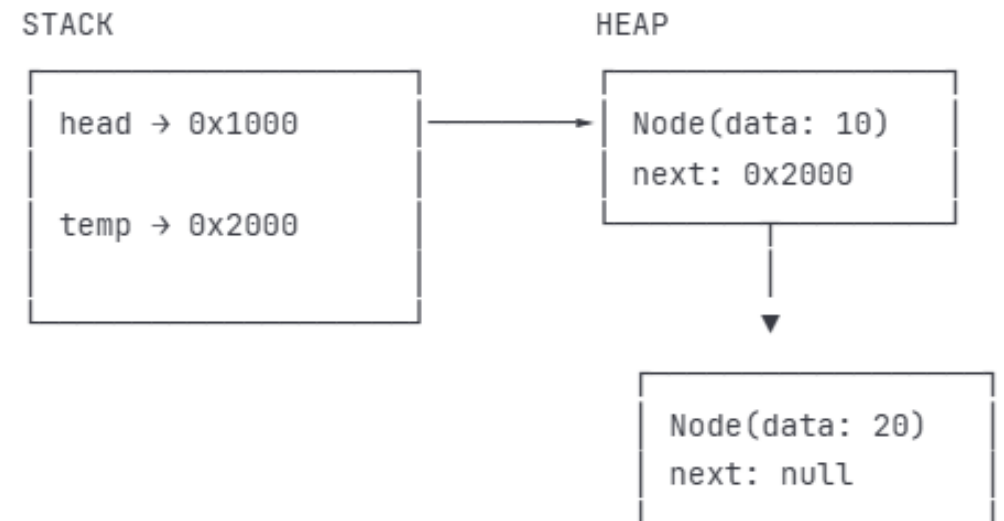
```
Node head = new Node();  
head.data = 10;  
head.next = null;
```



Adding Nodes to a SLL

- Step 2 : Create second node and attach

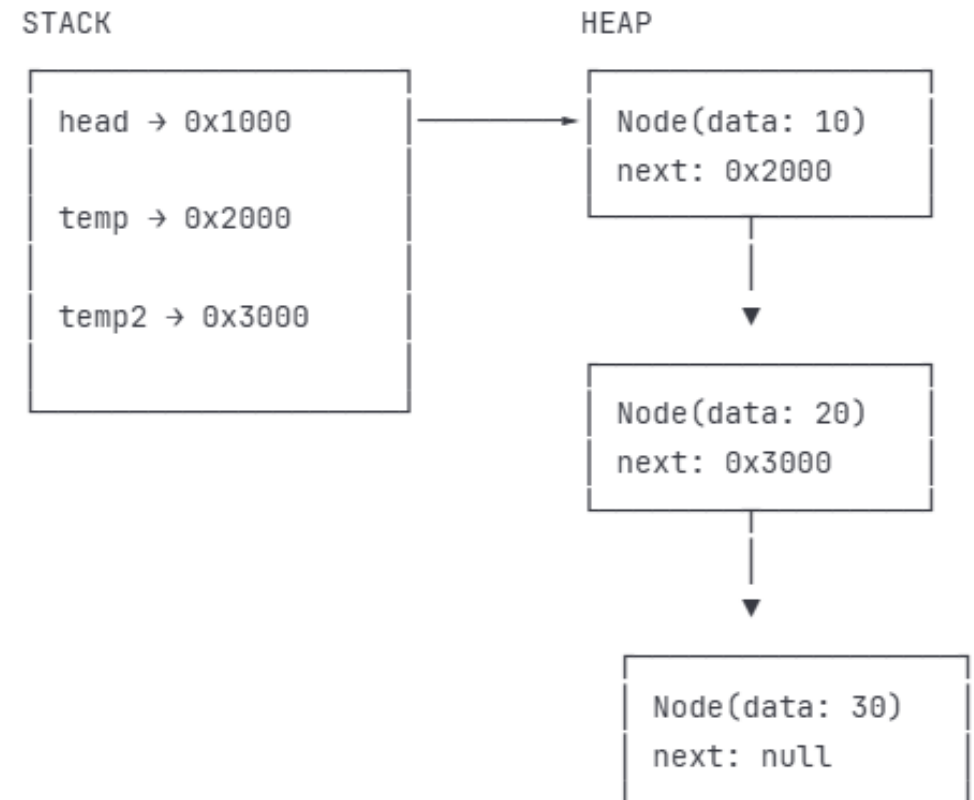
```
Node temp = new Node();  
temp.data = 20;  
temp.next = null;  
head.next = temp; // Link first node to second
```



Adding Nodes to a SLL

- Step 3: Create and attach third node

```
Node temp2 = new Node();  
temp2.data = 30;  
temp2.next = null;  
head.next.next = temp2; // Link second node to third
```



Core Operations on a Linked List

1. Insert at Beginning ($O(1)$)

****Real-life:**** Adding a new engine to the front of the train.

****Diagram:****

Before: head -> [20|●] -> [30|null]

After: head -> [10|●] -> [20|●] -> [30|null]

****Code:****

```
public void insertAtBeginning(int data) {  
    Node newNode = new Node(data); // Create new node  
    newNode.next = head;           // New node points to current head  
    head = newNode;                // Head now points to new node  
}
```

****Step-by-step:****

1. Create new node with data
2. Make new node point to current head
3. Update head to point to new node

Core Operations on a Linked List

```
1 ##### 2. Insert at End (O(n))
2
3 **Real-life:** Adding a new coach at the end of the train. You need to walk through all coaches first!
4
5 **Diagram:**
6 Before: head -> [10|•] -> [20|null]
7 After:  head -> [10|•] -> [20|•] -> [30|null]
8
9 **Code:**
10
11 public void insertAtEnd(int data) {
12     Node newNode = new Node(data);
13
14     // Special case: empty list
15     if (head == null) {
16         head = newNode;
17         return;
18     }
19
20     // Traverse to the last node
21     Node current = head;
22     while (current.next != null) {
23         current = current.next;
24     }
25
26     // Add new node at the end
27     current.next = newNode;
28 }
29 **Why O(n)?** We must traverse the entire list to find the last node!
30
```

Core Operations on a Linked List

3. Insert at Position ($O(n)$)

****Real-life:**** Inserting a coach between Coach 2 and Coach 3.

****Diagram:****

Before: head -> [10|●] -> [20|●] -> [40|null]
After: head -> [10|●] -> [20|●] -> [30|●] -> [40|null]
(inserted at position 2)

```
1  **Code:**
2  public void insertAtPosition(int data, int position) {
3      Node newNode = new Node(data);
4
5      // Special case: insert at beginning
6      if (position == 0) {
7          newNode.next = head;
8          head = newNode;
9          return;
10     }
11
12     // Traverse to position-1
13     Node current = head;
14     for (int i = 0; i < position - 1 && current != null; i++) {
15         current = current.next;
16     }
17
18     // If position is valid
19     if (current != null) {
20         newNode.next = current.next;
21         current.next = newNode;
22     } else {
23         System.out.println("Position out of bounds!");
24     }
25 }
```

Core Operations on a Linked List

```
#### 4. Delete from Beginning (O(1))
**Real-life:** Detaching the first coach.

**Diagram:**

Before: head -> [10|●] -> [20|●] -> [30|null]
After:  head -> [20|●] -> [30|null]

**Code:**

public void deleteFromBeginning() {
    if (head == null) {
        System.out.println("List is empty!");
        return;
    }

    head = head.next; // Simply move head to next node
    // The old first node will be garbage collected
}
```

Core Operations on a Linked List

```
1 ##### 5. Delete from End (O(n))
2 **Real-life:** Removing the last coach. You need to reach the second-to-last coach!
3
4 **Diagram:**
5 Before: head -> [10|•] -> [20|•] -> [30|null]
6 After:  head -> [10|•] -> [20|null]
7
8 **Code:**
9
10 public void deleteFromEnd() {
11     if (head == null) {
12         System.out.println("List is empty!");
13         return;
14     }
15
16     // Special case: only one node
17     if (head.next == null) {
18         head = null;
19         return;
20     }
21
22     // Traverse to second-to-last node
23     Node current = head;
24     while (current.next.next != null) {
25         current = current.next;
26     }
27
28     current.next = null; // Remove the last node
29 }
```

Core Operations on a Linked List

```
1 ##### 6. Delete by Value (O(n))
2 **Real-life:** Remove the coach carrying hazardous material!
3
4 **Code:**
5 public void deleteByValue(int value) {
6     if (head == null) {
7         System.out.println("List is empty!");
8         return;
9     }
10
11     // Special case: head contains the value
12     if (head.data == value) {
13         head = head.next;
14         return;
15     }
16
17     // Find the node before the one to delete
18     Node current = head;
19     while (current.next != null && current.next.data != value) {
20         current = current.next;
21     }
22
23     // If value found
24     if (current.next != null) {
25         current.next = current.next.next; // Skip the node to delete
26     } else {
27         System.out.println("Value not found!");
28     }
29 }
```

Core Operations on a Linked List

```
✓ ##### 7. Search (O(n))

**Code:**

public boolean search(int value) {
    Node current = head;
    while (current != null) {
        if (current.data == value) {
            return true;
        }
        current = current.next;
    }
    return false;
}
```

Core Operations on a Linked List

```
#### 8. Display/Traverse (O(n))

**Code:**

public void display() {
    if (head == null) {
        System.out.println("List is empty!");
        return;
    }

    Node current = head;
    System.out.print("List: ");
    while (current != null) {
        System.out.print(current.data + " -> ");
        current = current.next;
    }
    System.out.println("null");
}
```